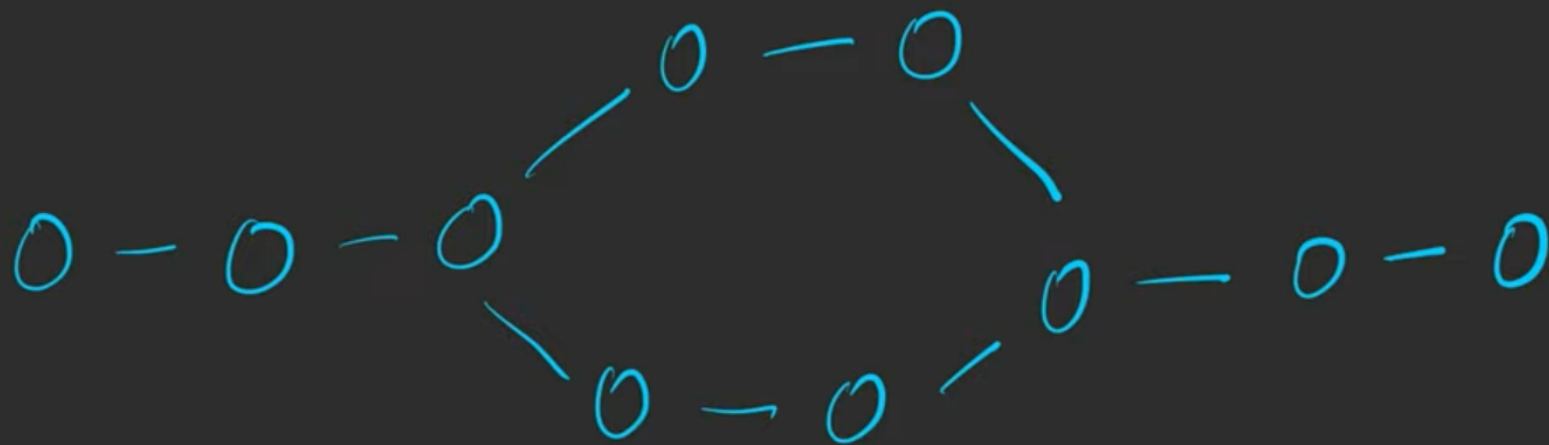


color the graph
with 2 colors such that
no adjacent nodes
have same color.

Bipartite Graph

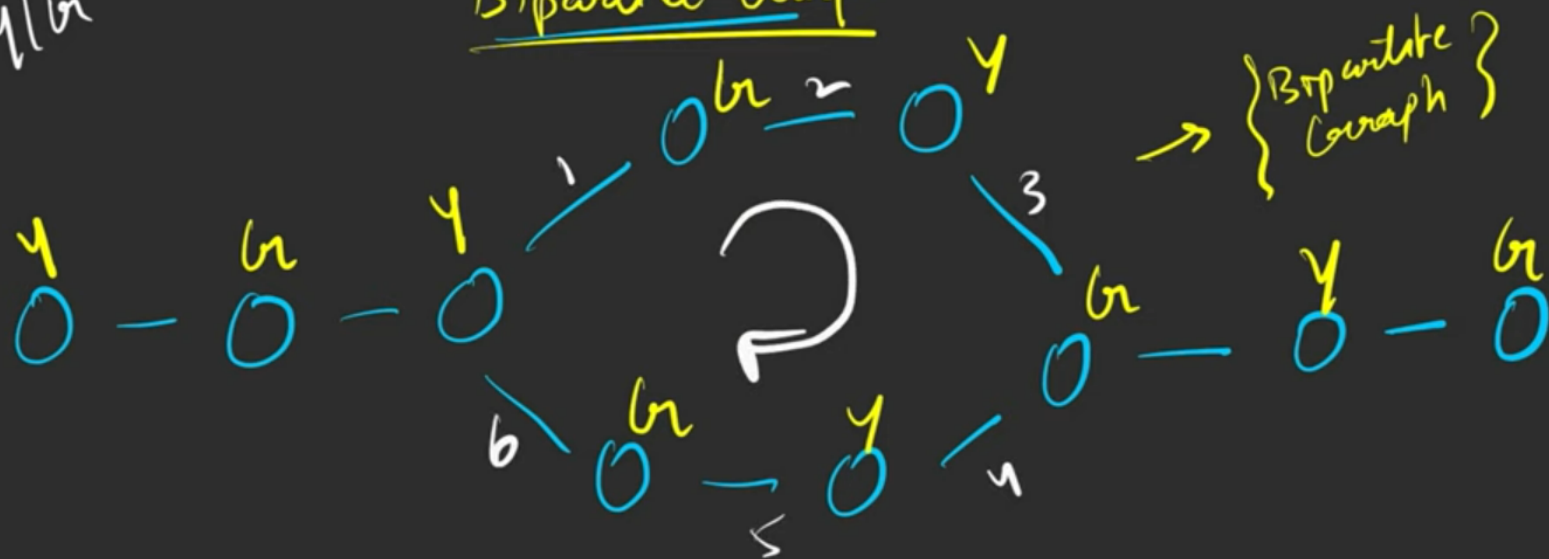




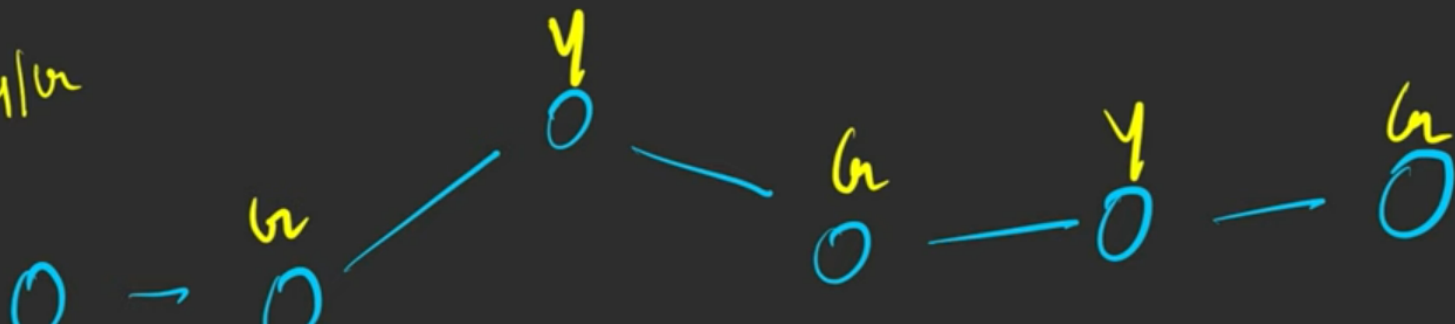
no ~ I have

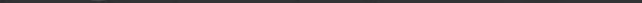
y/b

Bipartite Graph

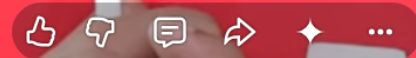


y/b

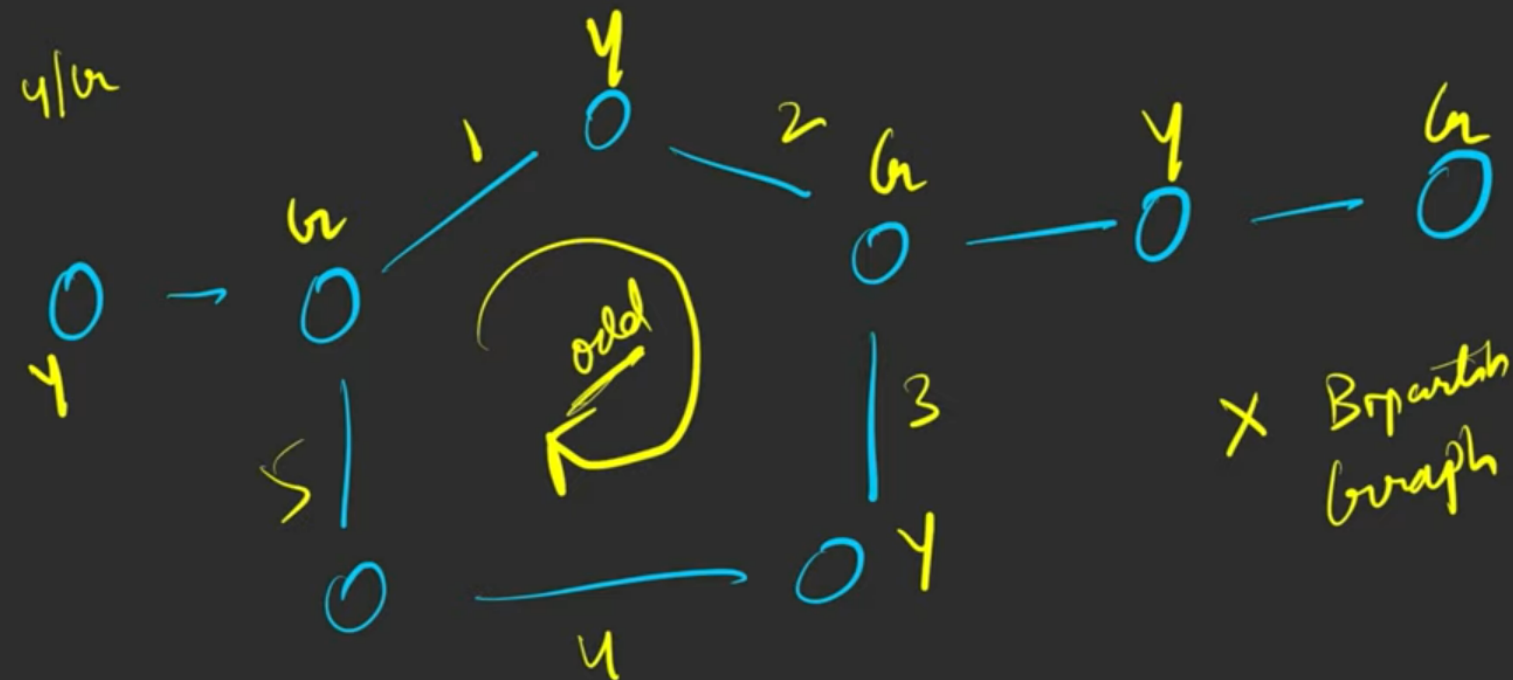




→ ✓ Bp intake



1.60



1.60

even
cycle
length

odd length
cycle

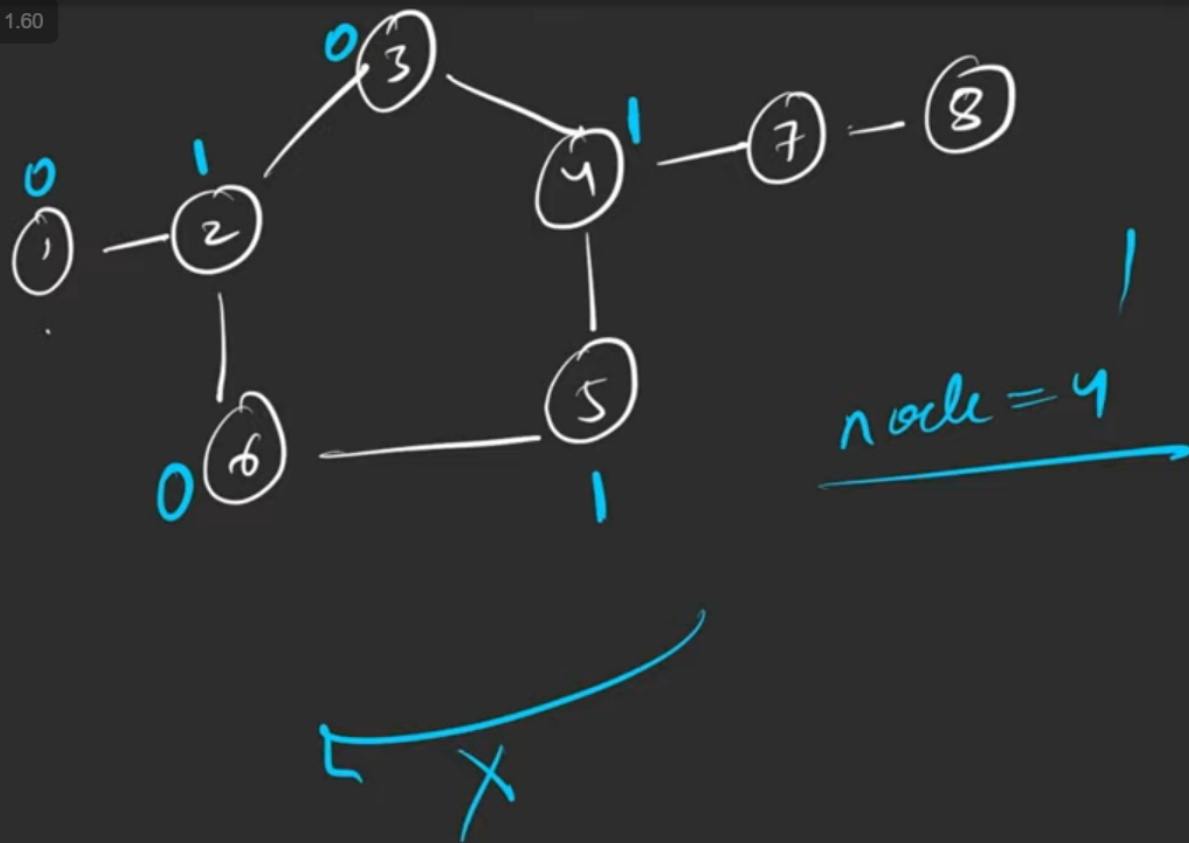


X

Bipartite



1.60



9

$1 \rightarrow \{2\}$
 $2 \rightarrow \{1, 3, 6\}$
 $3 \rightarrow \{2, 4\}$
 $4 \rightarrow \{3, 5, 7\}$
 $5 \rightarrow \{4, 6\}$



1.60



color

1	2	3	4	5	6	7	8
0	1	0	1	0	0	-1	-1

1 → 1
2 → {1, 3, 6}
3 → {2, 4}
4 → {3, 5, 7}
5 → {4, 6}
6 → {2, 5}
7 → {4, 8}
8 → {7}

106 of 132



```

private boolean check(int start, int V,
    ArrayList<ArrayList<Integer>>adj, int color[]) {
    Queue<Integer> q = new LinkedList<Integer>();
    q.add(start);
    color[start] = 0;
    while(!q.isEmpty()) {
        int node = q.peek();
        q.remove();

        for(int it : adj.get(node)) {
            // if the adjacent node is yet not colored
            // you will give the opposite color of the node
            if(color[it] == -1) {
                color[it] = 1 - color[node];
                q.add(it);
            }
            // is the adjacent guy having the same color
            // someone did color it on some other path
            else if(color[it] == color[node]) {
                return false;
            }
        }
    }
    return true;
}

public boolean isBipartite(int V, ArrayList<ArrayList<Integer>>adj)
{
    int color[] = new int[V];
    for(int i = 0; i < V; i++) color[i] = -1;

    for(int i = 0; i < V; i++) {
        if(color[i] == -1) {
            if(check(i, V, adj, color) == false) {
                return false;
            }
        }
    }
    return true;
}

```

```

1 // } Driver Code Ends
2 class Solution {
3     // colors a component
4     private:
5     bool check(int start, int V, vector<int>adj[], int color[]) {
6         queue<int> q;
7         q.push(start);
8         color[start] = 0;
9         while(!q.empty()) {
10             int node = q.front();
11             q.pop();
12
13             for(auto it : adj[node]) {
14                 // if the adjacent node is yet not colored
15                 // you will give the opposite color of the node
16                 if(color[it] == -1) {
17                     color[it] = !color[node];
18                     q.push(it);
19                 }
20                 // is the adjacent guy having the same color
21                 // someone did color it on some other path
22                 else if(color[it] == color[node]) {
23                     return false;
24                 }
25             }
26         }
27         return true;
28     }
29 public:
30     bool isBipartite(int V, vector<int>adj[]){
31         int color[V];
32         for(int i = 0; i < V; i++) color[i] = -1;
33
34         for(int i = 0; i < V; i++) {
35             if(color[i] == -1) {
36                 if(check(i, V, adj, color) == false) {
37                     return false;
38                 }
39             }
40         }
41         return true;
42     }
43 }

```




```

private boolean check(int start, int V,
    ArrayList<ArrayList<Integer>>adj, int color[]) {

    Queue<Integer> q = new LinkedList<Integer>();
    q.add(start);
    color[start] = 0;
    while(!q.isEmpty()) {
        int node = q.peek();
        q.remove();

        for(int it : adj.get(node)) {
            // if the adjacent node is yet not colored
            // you will give the opposite color of the node
            if(color[it] == -1) {

                color[it] = 1 - color[node];
                q.add(it);
            }
            // is the adjacent guy having the same color
            // someone did color it on some other path
            else if(color[it] == color[node]) {
                return false;
            }
        }
    }
    return true;
}

public boolean isBipartite(int V, ArrayList<ArrayList<Integer>>adj)
{
    int color[] = new int[V];
    for(int i = 0; i < V; i++) color[i] = -1;

    for(int i = 0; i < V; i++) {
        if(color[i] == -1) {
            if(check(i, V, adj, color) == false) {
                return false;
            }
        }
    }
    return true;
}

```

```

13- while(!q.empty()) {
14-     int node = q.front();
15-     q.pop();
16-
17-     for(auto it : adj[node]) {
18-         // if the adjacent node is yet not colored
19-         // you will give the opposite color of the node
20-         if(color[it] == -1) {
21-
22-             color[it] = !color[node];
23-             q.push(it);
24-         }
25-         // is the adjacent guy having the same color
26-         // someone did color it on some other path
27-         else if(color[it] == color[node]) {
28-             return false;
29-         }
30-     }
31- }
32- return true;
33- }
34- public:
35- bool isBipartite(int V, vector<int>adj[]){
36-     int color[V];
37-     for(int i = 0; i < V; i++) color[i] = -1;
38-
39-     for(int i = 0; i < V; i++) {
40-         if(color[i] == -1) {
41-             if(check(i, V, adj, color) == false) {
42-                 return false;
43-             }
44-         }
45-     }
46-     return true;
47- }
48-
49- };
50- // } Driver Code Ends

```



Compile & Run